ЛАБОРАТОРНА РОБОТА № 1

Тема: ПРИХОВУВАННЯ ДАНИХ У ЗОБРАЖЕННІ МЕТОДОМ ЗАМІНИ НАЙМЕНШОГО ЗНАЧУЩОГО БІТА (LSB)

Мета: дослідити принципи роботи методу LSB на основі навчальної програми CrypTool 2; проаналізувати ємність стеганографічного приховування даних; реалізувати алгоритм LSB для приховування текстових даних у зображенні, а також функціонал для вилучення прихованого тексту з зображення.

Хід роботи:

Завдання 1

Виконати приховування повідомлення у зображенні методом LSB на основі навчальної програми CrypTool 2 (Templates⇒Steganography⇒Image Steganography with LSB).

1. Завантажте та встановіть програмне забезпечення CrypTool 2 із офіційного сайту https://www.cryptool.org/en/ct2/downloads/. CrypTool 2 (CT2) – це безкоштовна програма з відкритим вихідним кодом для Windows, призначена для навчання криптографії та криптоаналізу, а також стеганографії.

2. Відкрийте проект Image Steganography with LSB (рис. 1.1), введіть своє ім’я та прізвище у поле для секретного повідомлення. Також, у відповідному полі, оберіть зображення, що буде слугувати контейнером (формат рекомендується обрати BMP або PNG (тільки не з прозорим фоном)).

3. Запустіть програму на виконання (натисніть Play у головному меню програми) та оберіть кількість бітів для модифікації за допомогою опції Choose bits for hiding process. Оберіть один найменш значущий біт та настаніть Apply changes (рис. 1.2). Після цього дані вбудуються у контейнер та згенерується нове зображення зі стеганографічними даними (додайте скріншот до звіту).

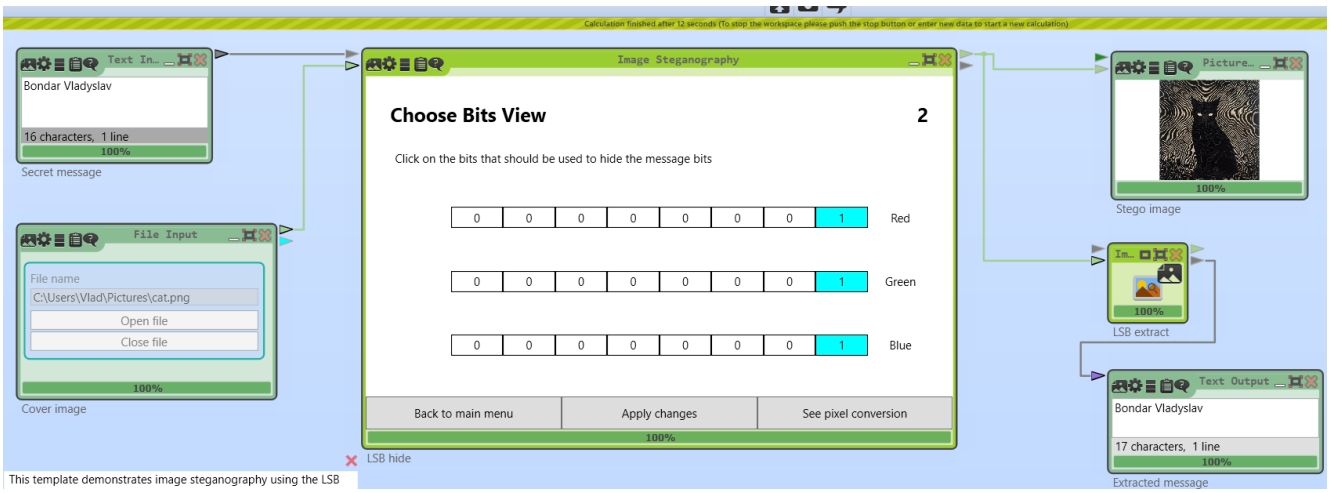


Рис. 1. - Зображення зі стеганографічними даними

4. Натисніть опцію See pixel conversation та перегляньте процес заміни пікселів. Детально опишіть у звіті приховування методом LSB першої та останньої літери секретного повідомлення (зі скріншотами). Зразок наведено нижче. Зауваження: у CrypTool 2 біти символу записуються у LSB компонент пікселів у форматі little-endian (в зворотному порядку)

Перша літера	***Остання літера***
B = 66 → 01000010	v = 118 → 01110110
В зворотному порядку 010 000 10	В зворотному порядку 011 011 10
Модифікація пікселя (0, 0)	***Модифікація пікселя (40, 0)***



RGB bits before	RGB bits after	RGB bits before	RGB bits after
Red value: 01010111	Red value: 01010110	Red value: 00100110	Red value: 00100110
Green value: 01010111	Green value: 01010111	Green value: 00100110	Green value: 00100111
Blue value: 01011001	Blue value: 01011000	Blue value: 00101000	Blue value: 00101001

Модифікація пікселя (1, 0)



RGB bits before

Red value: 11000110
Green value: 11000110
Blue value: 11001000



RGB bits after

Red value: 11000110
Green value: 11000110
Blue value: 11001000

Модифікація пікселя (41, 0)



RGB bits before

Red value: 00100110
Green value: 00100110
Blue value: 00101000



RGB bits after

Red value: 00100110
Green value: 00100111
Blue value: 00101001

Модифікація пікселя (2, 0)



RGB bits before

Red value: 01110010
Green value: 01110010
Blue value: 01110100



RGB bits after

Red value: 01110011
Green value: 01110010
Blue value: 01110101

Модифікація пікселя (42, 0)



RGB bits before

Red value: 00100101
Green value: 00100101
Blue value: 00100111



RGB bits after

Red value: 00100101
Green value: 00100100
Blue value: 00100111

Перша літера секретного повідомлення «B» (ASCII-код 66) у двійковому форматі записується як 01000010. Програма CrypTool 2 застосовує формат little-endian, тому бінарний код читається у зворотному порядку від молодшого біта до старшого як 01000010. Остання шістнадцята літера повідомлення «v» (ASCII-код 118) записується у двійковому вигляді як 01110110, що реверсується у 01101110. За цими бітовими послідовностями алгоритм послідовно змінює найменш значущі біти каналів червоного, зеленого та синього кольорів обраних пікселів. Модифікація відбувається шляхом простої заміни: якщо поточний біт повідомлення є одиницею, то відповідний молодший біт каналу кольору пікселя також встановлюється як одиниця, а якщо нулю – перезаписується як нуль.

5. За допомогою опції Overview and Hiding Capacity Information перегляньте інформацію про довжину повідомлення та ємність приховування (додайте скріншот до звіту). Самостійно обчисліть довжину повідомлення та максимальну ємність приховування, якщо замінювати 1 найменш значущий біт. Порівняйте власноруч обчислені значення із тими, що на скріншоті (рис. 1.3).

Повідомлення Bondar Vladyslav містить 16 символів (включно з пробілом). Кожен символ кодується у 8 бітів. Таким чином довжина повідомлення:

$16 * 8 = 128$ бітів.

Зображення розміром 1000x1002. Тоді максимальна ємність приховування, якщо замінювати 1 найменш значущий біт буде:

$C = 1000 * 1002 * 3 * 1 = 3\ 006\ 000$ бітів.

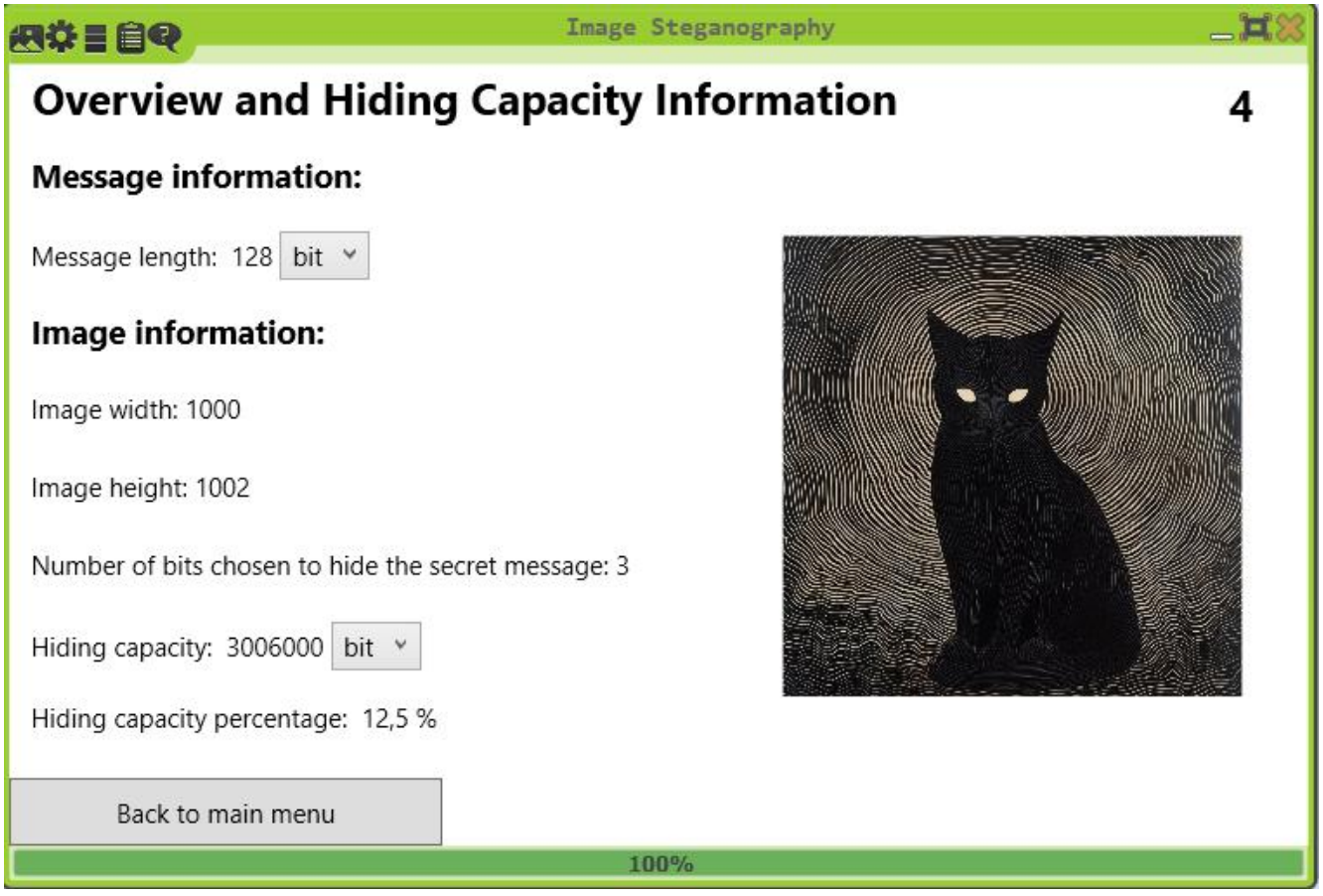


Рис. 2. - Інформація про довжину повідомлення та ємність приховування

6. Забезпечте збереження стегоконтейнеру до нового файлу. Для цього додайте до проекту компонент File Output, використовуючи розділ Tools на панелі модульних компонентів Components. Ця панель розміщена ліворуч робочого вікна та дає змогу додавати до проекту нові складові, таким чином удосконалюючи його.

Встановіть за допомогою стрілок зв'язок між компонентами програми (рис. 1.4). У налаштуваннях File Output вкажіть ім'я нового файлу, його розширення та шлях. Збережіть проект, використовуючи кнопку Save.

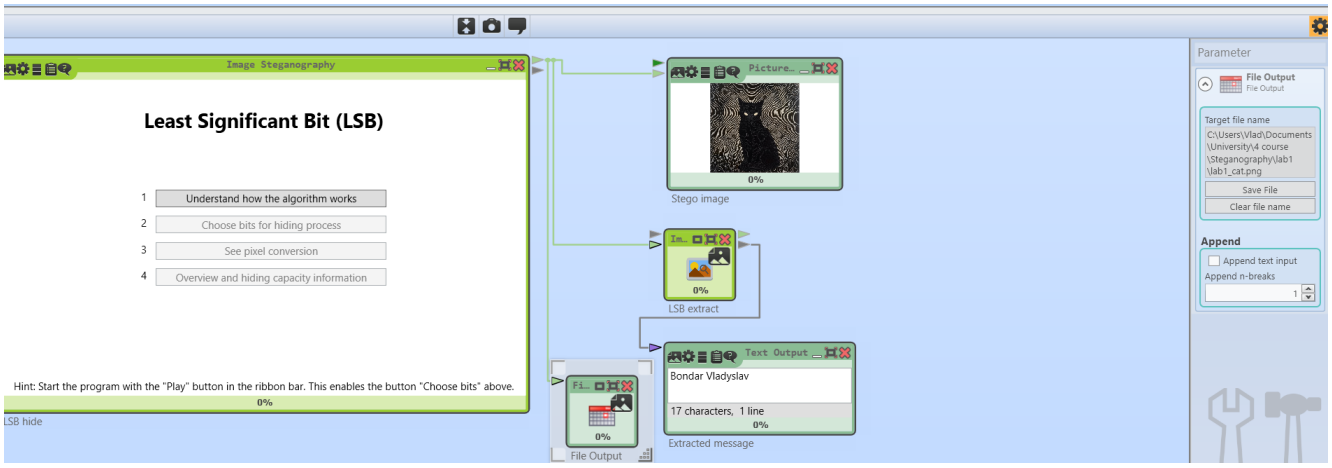


Рис. 3. - Додавання до проекту компоненту File Output та збереження файлу

7. Знову запустіть програму на виконання. Порівняйте у звіті оригінальне зображення та зображення із прихованими даними (рис. 1.5, рис. 1.6).

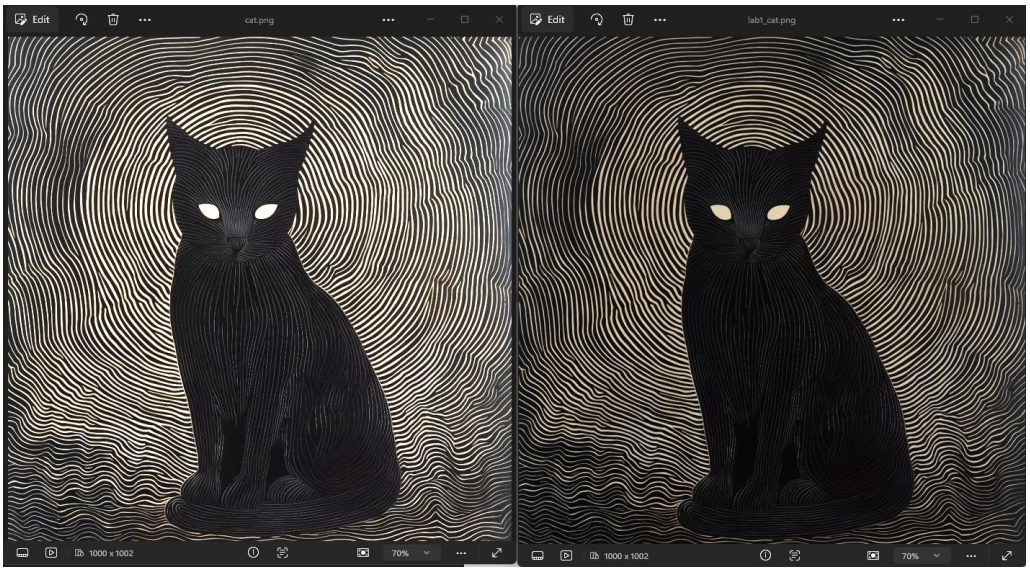


Рис. 4. – Обидва зображення. Зліва оригінальне, а справа з прихованими даними

Візуальне порівняння оригінального файлу та отриманого стегоконтейнера підтверджує абсолютну ідентичність зображень для людського ока. Це практично доводить, що заміна найменш значущого біта дозволяє ефективно приховувати дані без помітної деградації якості носія.

8. Обміняйтеся зображенням із вбудованими даними із іншим студентом своєї групи. Додайте отримане зображення до звіту.

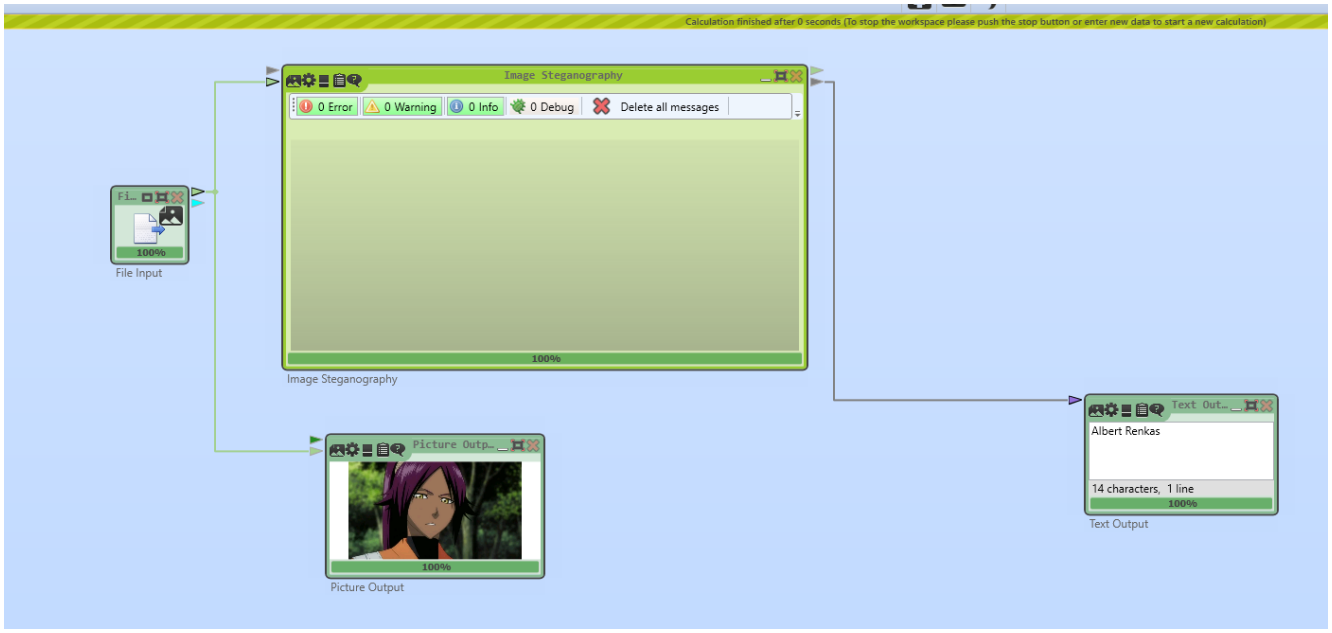


Рис. 5. – Зображенням із вбудованими даними іншого студента

9. Створіть новий проект для вилучення прихованого повідомлення, наприклад під назвою LSB_dec.cwm. Додайте до проекту компоненти File Input, Image Steganography та Text Output. Встановіть зв’язки, налаштуйте параметри роботи складових проекту як показано на рисунку (рис. 1.5).

10. Збережіть проект та перевірте його роботу. Результатом правильного виконання буде коректно вилучене повідомлення (рис. 1.6). Додайте скріншот проекту до звіту.

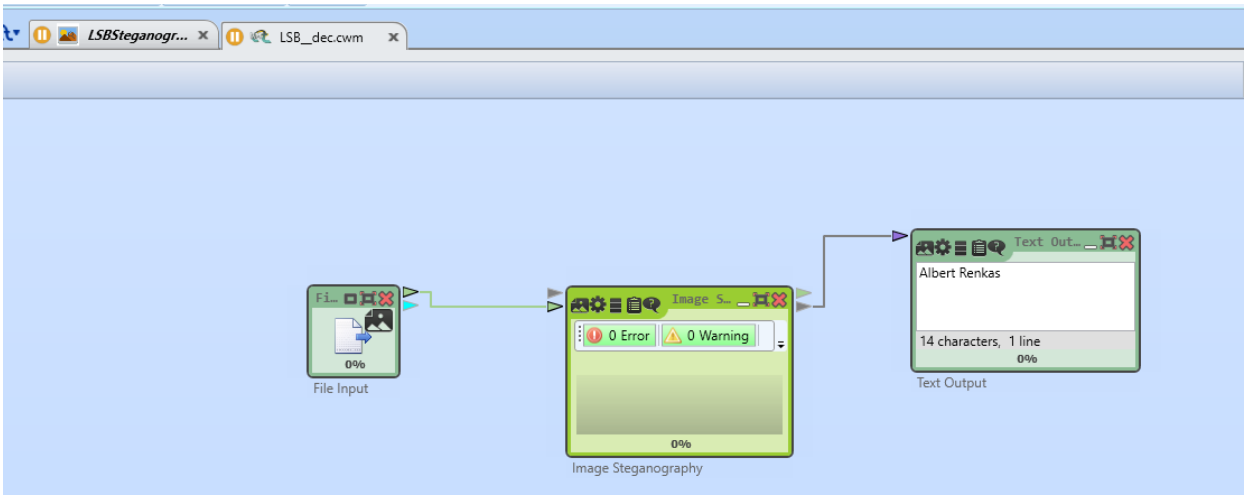


Рис. 6. – Перевірка роботи проєкту на зображенні іншого студента

Завдання 2

Розробити програму мовою програмування (Python, C++, Java тощо) для приховування текстових даних у зображенні методом LSB. Реалізувати функціонал для вилучення прихованого тексту із зображення. Виконати порівняння оригінального зображення та зображення із прихованими даними. Реалізувати інтерфейс взаємодії з користувачем – пропонувати вибір між приховуванням та вилученням тексту, запитувати текст для приховування та шлях до зображення. Забезпечити роботу із зображеннями у форматі PNG або BMP. Перевірити коректність роботи програми на різних зображеннях.

Код:

```
import os
from PIL import Image

def _embed_bits(image: Image.Image, binary_message: str):
    pixel_map = image.load()
    binary_index = 0
    message_len = len(binary_message)

    for y in range(image.height):
        for x in range(image.width):
            if binary_index >= message_len:
                return

            r, g, b = pixel_map[x, y]  # type: ignore
```

```
            r = (r & ~1) | int(binary_message[binary_index])
            binary_index += 1

            if binary_index < message_len:
                g = (g & ~1) | int(binary_message[binary_index])
                binary_index += 1

            if binary_index < message_len:
                b = (b & ~1) | int(binary_message[binary_index])
                binary_index += 1

            pixel_map[x, y] = (r, g, b)  # type: ignore

def encode_image(image_path: str, message: str, output_path: str):
    full_message = message + "END"
    binary_message = ''.join(format(ord(char), '08b') for char in full_message)

    image = Image.open(image_path).convert('RGB')
    max_capacity_bits = image.width * image.height * 3

    if len(binary_message) > max_capacity_bits:
        raise ValueError(
            f"Повідомлення завелике! Доступно: {max_capacity_bits} біт, "
            f"потрібно: {len(binary_message)} біт."
        )

    _embed_bits(image, binary_message)

    image.save(output_path)
    print(f"\n[Успіх] Повідомлення успішно вбудовано та збережено у: {output_path}")

def _extract_text(image: Image.Image) -> str:
    pixel_map = image.load()
    binary_data = ""
    decoded_chars = []

    for y in range(image.height):
        for x in range(image.width):
            r, g, b = pixel_map[x, y]  # type: ignore

            binary_data += f"{r & 1}{g & 1}{b & 1}"

            while len(binary_data) >= 8:
                byte = binary_data[:8]
```

```
                binary_data = binary_data[8:]

                decoded_chars.append(chr(int(byte, 2)))

                if "".join(decoded_chars[-3:]) == "END":
                    return "".join(decoded_chars[:-3])

    raise ValueError("Маркер завершення 'END' не знайдено або повідомлення відсутнє.")

def decode_image(image_path: str) -> str:
    image = Image.open(image_path).convert('RGB')
    return _extract_text(image)

def main():
    while True:
        print("\n" + "=" * 20 + " LSB Steganography " + "=" * 20)
        print("1. Приховати повідомлення")
        print("2. Вилучити повідомлення")
        print("3. Вихід")
        choice = input("Оберіть опцію: ").strip()

        if choice == '1':
            img_path = input("Введіть шлях до вхідного зображення (PNG/BMP): ").strip().strip('"')
            msg = input("Введіть текст для приховування: ")
            out_path = input("Введіть шлях для збереження стегозображення: ").strip().strip('"')
            try:
                encode_image(img_path, msg, out_path)
            except Exception as e:
                print(f"[Помилка]: {e}")

        elif choice == '2':
            img_path = input("Введіть шлях до стегозображення: ").strip().strip('"')
            try:
                extracted = decode_image(img_path)
                print(f"\n[Вилучене повідомлення]: {extracted}")
            except Exception as e:
                print(f"[Помилка]: {e}")

        elif choice == '3':
            print("Роботу завершено.")
            break
        else:
            print("Невірний вибір. Спробуйте ще раз.")

if __name__ == "__main__":
```

```
main()
```

```
Problems  Output  Debug Console  Terminal  Ports  SonarQube

PS C:\Users\Vlad\Documents\University\4 course\Steganography\lab1>  & 'C:\Users\Vlad\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c

 ================== LSB Steganography ==================
 1. Приховати повідомлення
 2. Вилучити повідомлення
 3. Вихід
 Оберіть опцію: 1
 Введіть шлях до вхідного зображення (PNG/BMP): C:\Users\Vlad\Pictures\cat_1_png.png
 Введіть текст для приховування: First image. Python program.
 Введіть шлях для збереження стегозображення: C:\Users\Vlad\Documents\University\4 course\Steganography\lab1\cat_1_stego.png

 [Успіх] Повідомлення успішно вбудовано та збережено у: C:\Users\Vlad\Documents\University\4 course\Steganography\lab1\cat_1_stego.png

 ================== LSB Steganography ==================
 1. Приховати повідомлення
 2. Вилучити повідомлення
 3. Вихід
 Оберіть опцію: 1
 Введіть шлях до вхідного зображення (PNG/BMP): C:\Users\Vlad\Pictures\car_png.png
 Введіть текст для приховування: Second image. Python program.
 Введіть шлях для збереження стегозображення: C:\Users\Vlad\Documents\University\4 course\Steganography\lab1\car_stego.png

 [Успіх] Повідомлення успішно вбудовано та збережено у: C:\Users\Vlad\Documents\University\4 course\Steganography\lab1\car_stego.png

 ================== LSB Steganography ==================
 1. Приховати повідомлення
 2. Вилучити повідомлення
 3. Вихід
 Оберіть опцію: 2
 Введіть шлях до стегозображення: C:\Users\Vlad\Documents\University\4 course\Steganography\lab1\cat_1_stego.png

 [Вилучене повідомлення]: First image. Python program.

 ================== LSB Steganography ==================
 1. Приховати повідомлення
 2. Вилучити повідомлення
 3. Вихід
 Оберіть опцію: 2
 Введіть шлях до стегозображення: C:\Users\Vlad\Documents\University\4 course\Steganography\lab1\car_stego.png

 [Вилучене повідомлення]: Second image. Python program.

 ================== LSB Steganography ==================
 1. Приховати повідомлення
 2. Вилучити повідомлення
 3. Вихід
 Оберіть опцію:
```

Рис. 7. – Результат роботи програми



Рис. 8. – Порівняння зображень Зліва оригінальне, а справа з прихованими даними

Для написання власної програми я обрав мову Python, оскільки вона має дуже зручну бібліотеку Pillow для роботи із зображеннями. Щоб код працював швидко і не перевантажував оперативну пам'ять, я використав сучасний метод load, який дає прямий доступ до кольорів кожного пікселя.

Логіка програми поділена на дві основні функції. Функція приховування спочатку дописує до нашого таємного тексту слово «END» – це своєрідний маркер, який потім підкаже програмі, де саме закінчується повідомлення і починається просто пусте тло картинки. Далі весь цей текст перетворюється на звичайний двійковий код. Після цього алгоритм проходить по кожному пікселю і по черзі акуратно замінює найменш значущий біт у червоному, зеленому та синьому каналах кольору на біти нашого тексту. Оскільки змінюється лише останній біт, колір пікселя зсувається максимум на одиницю, що абсолютно непомітно для ока.

Функція вилучення робить усе навпаки. Вона так само йде по пікселях з самого початку, збирає найменш значущі біти з кожного кольорового каналу і групує їх по вісім штук, щоб перетворити назад у звичайні букви. Цей процес перекладу триває безперервно, і після кожної нової літери програма перевіряє, чи не склалося з останніх трьох букв слово «END». Як тільки алгоритм бачить цей маркер, він зупиняється і видає нам розшифрований текст.

Взаємодія з програмою відбувається через просте консольне меню, де можна обрати потрібну дію та вказати шляхи до файлів. Тестування показало, що скрипт працює безвідмовно: текст успішно ховається і вилучається назад, а сама картинка візуально залишається такою ж, як і до обробки.

Git: *https://github.com/Student-Vlad-Bondar/Steganography-labs*

Контрольні запитання:

1. Що таке стеганографія і чим вона відрізняється від криптографії?

Стеганографія приховує сам факт існування або передачі секретного повідомлення, роблячи його невидимим усередині носія. Криптографія приховує

лише зміст повідомлення шляхом шифрування, проте факт наявності закритого трафіку залишається відкритим для сторонніх осіб.

2. Дайте визначення таким поняттям як контейнер, стегооб’єкт, стегодані, стеганографічний ключ, кодер, декодер, стегоканал, стегоаналіз.

Контейнер: цифровий об'єкт (зображення, аудіо тощо), у який вбудовується повідомлення.

Стегооб'єкт: контейнер, що вже містить у собі приховані дані.

Стегодані: безпосередньо приховувана інформація.

Стеганографічний ключ: секретний параметр або алгоритмічний ключ, необхідний для вбудовування чи правильного вилучення даних.

Кодер / Декодер: програмні модулі відповідно для інтеграції та вилучення стегоданих.

Стегоканал: цифрове або фізичне середовище передачі стегооб'єкта.

Стегоаналіз: наука та методологія виявлення прихованих повідомлень у контейнерах.

3. Назвіть основні складові стегосистеми.

Основні складові стегосистеми: повідомлення (стегодані), пустий контейнер, стеганографічний ключ, кодер (алгоритм вбудовування), стегоканал, декодер (алгоритм вилучення) та готовий стегооб'єкт.

4. Що таке метод заміни найменшого значущого біта (LSB), і як він реалізується у цифрових зображеннях?

Метод заміни найменшого значущого біта полягає у підміні молодших бітів значень колірних каналів (R, G, B) пікселів бітами секретного повідомлення.

5. Чому заміна найменших значущих бітів пікселів не впливає на візуальну якість зображення?

Молодший біт має найменшу вагу ($2^0 = 1$). Зміна числового значення інтенсивності кольору в діапазоні від 0 до 255 максимум на одиницю створює настільки мізерну різницю у відтінку, що людське око її фізіологічно не здатне помітити.

6. Які формати зображень найкраще підходять для реалізації методу LSB і чому?

Найкраще підходять формати без стиснення або зі стисненням без втрат (BMP, PNG). Формати зі стисненням із втратами (як JPEG) використовують дискретне косинусне перетворення та квантування, що безповоротно спотворює та перезаписує найменш значущі біти, знищуючи приховане повідомлення.

7. Як можна обчислити максимальну ємність зображення для приховування даних методом LSB

Максимальна ємність обчислюється за формулою: $C = W \times H \times 3 \times N$ (бітів), де W – ширина зображення, H – висота, 3 – канали кольору (R, G, B), а N – кількість змінюваних молодших бітів на кожен канал.

8. Як змінюються RGB-компоненти пікселів при приховуванні даних методом LSB?

Значення байта колірного каналу стає парним, якщо туди записується біт повідомлення «0», і непарним, якщо записується біт «1». При цьому оригінальне значення кольору змінюється щонайбільше на одиницю.

Висновки: Під час виконання лабораторної роботи досліджено алгоритми просторової стеганографії, зокрема метод заміни найменшого значущого біта (LSB) за допомогою спеціалізованого середовища CrypTool 2. Практичним результатом стала розробка власної консольної програми мовою Python (із застосуванням бібліотеки Pillow) для надійного вбудовування та вилучення текстових даних із зображень форматів PNG та BMP. У процесі реалізації алгоритму було успішно

проаналізовано механіку побітової модифікації RGB-каналів, верифіковано розрахунок максимальної ємності стегоконтейнера та впроваджено механізм ідентифікації меж прихованого тексту за допомогою спеціального маркера завершення («END»). Практичне тестування підтвердило ефективність застосованого методу: інтеграція даних відбувається без жодної візуальної деградації якості оригінального файлу.